Structured Data Management

Final Project

Submitted By:

NAMITA CHHANTYAL

DARLINGTON MASODZI

BILLY MUCHIPISI

Submission Date: 8th May, 2025

Table of Contents

Step 1: Business Problem Definition

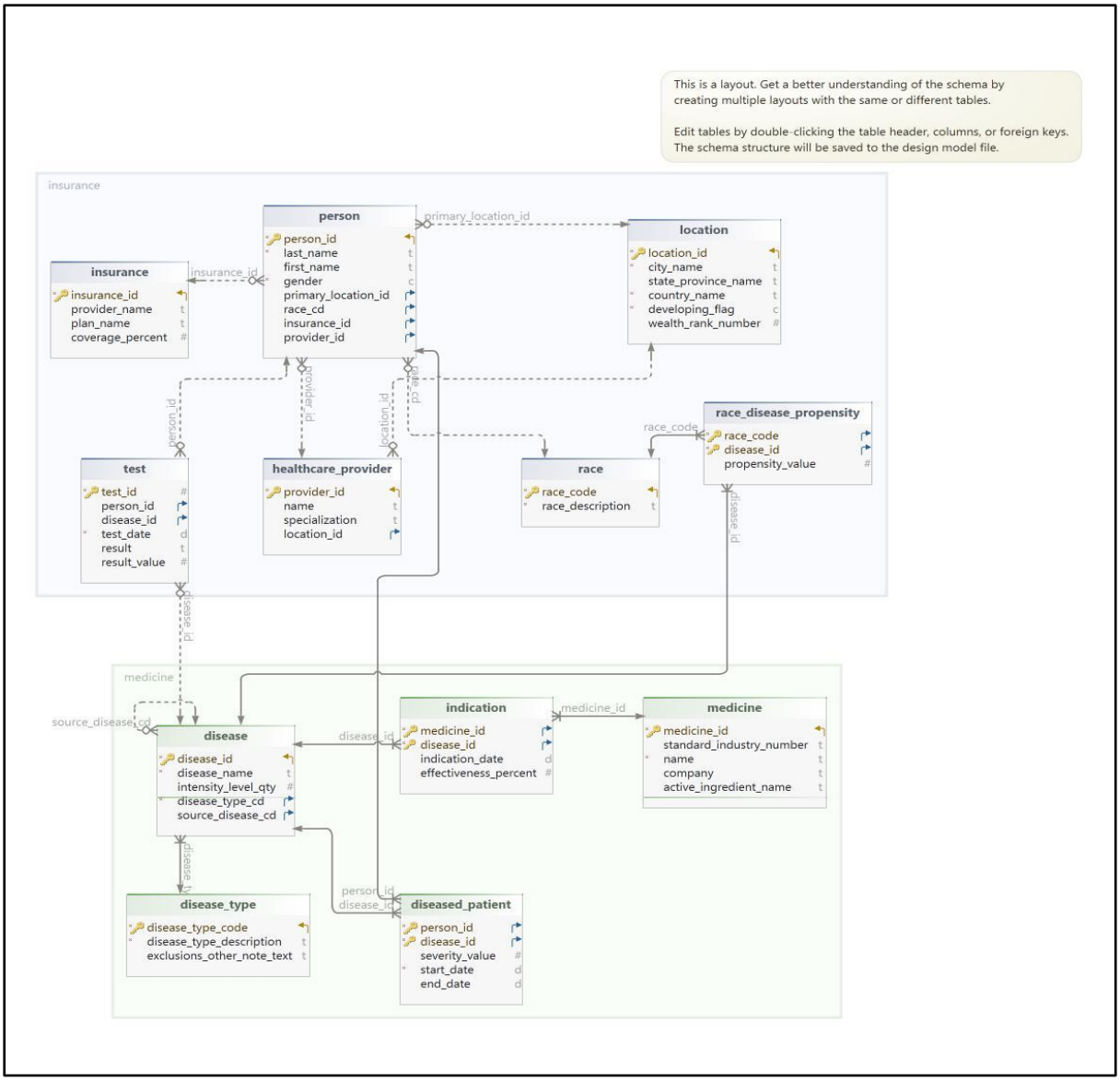
Goal: Improve public health outcomes through disease tracking, treatment effectiveness, patient demographics, and risk factor analysis.

Problem Statement

Public health agencies need a reliable database system to track diseases, assess racial and geographic disparities, evaluate medication effectiveness, and identify patterns for early intervention. This system should support operational decision-making and strategic analytics.

Step 2: Relational ERD & Data Dictionary

ER diagram:



This ER (Entity-Relationship) diagram represents a comprehensive Disease Data Management System used to analyze public health outcomes. The diagram is structured into three key components: Patient and Demographics, Healthcare and Disease Events, and Insurance and Medication Details. Below is an explanation of each section and the relationships between the entities.

1. Patient and Demographics Section:

This section includes entities that record patient information and demographic data:

- Person Table:
    - Attributes: person_id, last_name, first_name, gender, primary_location_id, race_cd, insurance_id, provider_id.
    - Represents individuals registered in the system with demographic details.
    - Relationships:
        - Linked to Location through primary_location_id.
        - Connected to Insurance and Healthcare Provider entities via insurance_id and provider_id.
        - Associated with the Race table via race_cd.
- Location Table:
    - Attributes: location_id, city_name, state_province_name, country_name, developing_flag, wealth_rank_number.
    - Represents the geographic data related to patients and providers.
- Race Table:
    - Attributes: race_code, race_description.
    - Stores ethnicity or racial background.
- Race Disease Propensity Table:
    - Attributes: race_code, disease_id, propensity_value.
    - Provides insight into the likelihood of certain diseases being prevalent among specific racial groups.

Key Insight:

This section focuses on capturing patient demographics and geographic data, crucial for analyzing disease trends and healthcare accessibility.

2. Healthcare and Disease Events Section:

This section contains information about healthcare providers, disease diagnosis, and testing:

- Healthcare Provider Table:
    - Attributes: provider_id, name, specialization, location_id.
    - Represents doctors or medical personnel responsible for diagnosis or treatment.
- Test Table:
    - Attributes: test_id, person_id, disease_id, test_date, result, result_value.
    - Stores the diagnostic tests taken by patients, linking to the Person and Disease tables.
- Disease Table:
    - Attributes: disease_id, disease_name, intensity_level_qty, disease_type_cd, source_disease_cd.
    - Represents information about different diseases, including intensity and type.
- Disease Type Table:
    - Attributes: disease_type_code, disease_type_description, exclusions_other_note_text.
    - Categorizes diseases by type, enabling better classification.
- Diseased Patient Table:
    - Attributes: person_id, disease_id, severity_value, start_date, end_date.
    - Tracks patients diagnosed with specific diseases and records the severity and duration of the illness.

Key Insight:

This section is pivotal for tracking medical events and healthcare provider roles in managing diseases.

3. Insurance and Medication Details Section:

This section tracks insurance plans and medication information:

- Insurance Table:
    - Attributes: insurance_id, provider_name, plan_name, coverage_percent.
    - Records the insurance details associated with each patient.
- Medicine Table:
    - Attributes: medicine_id, standard_industry_number, name, company, active_ingredient_name.
    - Stores details about prescribed medications.

- Indication Table:
    - Attributes: medicine_id, disease_id, indication_date, effectiveness_percent.
    - Links medications to diseases they treat, showing effectiveness percentages.

Key Insight:

This section aids in analyzing the effectiveness of treatments and understanding insurance coverage trends.

Relationships Between Tables:

- The Person table serves as the central entity for patient demographics, connecting to Healthcare Providers, Insurance, Tests, and Diseased Patient.
- The Disease table links to the Medicine and Indication tables, providing insights into treatment plans and medication effectiveness.
- Location acts as a shared reference for both patients and providers, helping to map healthcare delivery geographically.
- Race Disease Propensity bridges racial demographics and disease occurrences, highlighting the correlation between ethnicity and disease risk.

Key Features of the ER Diagram:

1. Comprehensive Data Coverage:
    - Captures demographic, healthcare, disease, insurance, and medication information in one model.
2. Logical Data Flow:
    - Clear connections between patients, healthcare providers, and diagnostic tests, enabling integrated data analysis.
3. Support for Analytical Insights:
    - Allows analysis of disease prevalence, treatment effectiveness, and patient demographics, making it suitable for healthcare research and policy-making.
4. Flexibility and Scalability:
    - The modular design allows for easy addition of new attributes or tables as healthcare data requirements evolve.

Use Cases for Healthcare Providers:

1. Disease Prevalence Tracking:
    - Analyzing how diseases spread across regions and demographic groups.

2. Healthcare Provider Efficiency:
    - Comparing test outcomes and treatment success rates among providers.
3. Insurance Utilization:
    - Identifying which insurance plans are most effective in covering critical treatments.
4. Medication Effectiveness:
    - Correlating medication types with disease recovery rates for evidence-based healthcare planning.

Data definition schemas are also created. The file name is FinalDataDictionary.html. Also the screenshots of the datadictionary are below:

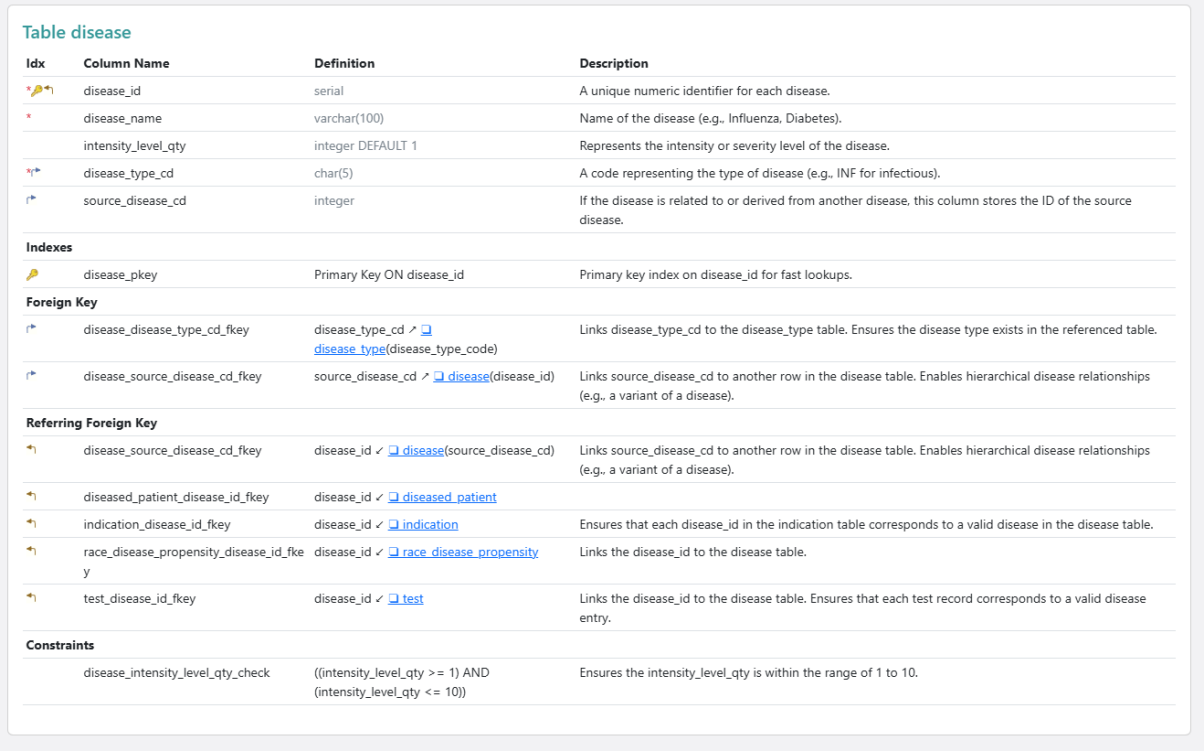


Table disease

Idx	Column Name	Definition	Description
*🔑↰	disease_id	serial	A unique numeric identifier for each disease.
*	disease_name	varchar(100)	Name of the disease (e.g., Influenza, Diabetes).
	intensity_level_qty	integer DEFAULT 1	Represents the intensity or severity level of the disease.
*↱	disease_type_cd	char(5)	A code representing the type of disease (e.g., INF for infectious).
↱	source_disease_cd	integer	If the disease is related to or derived from another disease, this column stores the ID of the source disease.
Indexes			
🔑	disease_pkey	Primary Key ON disease_id	Primary key index on disease_id for fast lookups.
Foreign Key			
↱	disease_disease_type_cd_fkey	disease_type_cd ↗ ❑ disease_type(disease_type_code)	Links disease_type_cd to the disease_type table. Ensures the disease type exists in the referenced table.
↱	disease_source_disease_cd_fkey	source_disease_cd ↗ ❑ disease(disease_id)	Links source_disease_cd to another row in the disease table. Enables hierarchical disease relationships (e.g., a variant of a disease).
Referring Foreign Key			
↰	disease_source_disease_cd_fkey	disease_id ✓ ❑ disease(source_disease_cd)	Links source_disease_cd to another row in the disease table. Enables hierarchical disease relationships (e.g., a variant of a disease).
↰	diseased_patient_disease_id_fkey	disease_id ✓ ❑ diseased_patient	
↰	indication_disease_id_fkey	disease_id ✓ ❑ indication	Ensures that each disease_id in the indication table corresponds to a valid disease in the disease table.
↰	race_disease_propensity_disease_id_fkey	disease_id ✓ ❑ race_disease_propensity	Links the disease_id to the disease table.
↰	test_disease_id_fkey	disease_id ✓ ❑ test	Links the disease_id to the disease table. Ensures that each test record corresponds to a valid disease entry.
Constraints			
	disease_intensity_level_qty_check	((intensity_level_qty >= 1) AND (intensity_level_qty <= 10))	Ensures the intensity_level_qty is within the range of 1 to 10.

Table disease_type

Idx	Column Name	Definition	Description
*🔑↰	disease_type_code	char(5)	Represents a unique identifier for the disease type (e.g., INF for infectious).
*	disease_type_description	varchar(1000)	A detailed description of the disease type (e.g., "Infectious diseases caused by pathogens").
	exclusions_other_note_text	varchar(2000)	Contains any special notes or exclusions related to this disease type.
Indexes			
🔑	disease_type_pkey	Primary Key ON disease_type_code	A unique code that identifies the type of disease.
Referring Foreign Key			
↰	disease_disease_type_cd_fkey	disease_type_code ✓ ❑ disease(disease_type_cd)	Links disease_type_cd to the disease_type table. Ensures the disease type exists in the referenced table.

Table diseased_patient

Idx	Column Name	Definition	Description
*🔑↱	person_id	integer	Represents the unique identifier of a patient.
*🔑↱	disease_id	integer	A unique numeric identifier for each disease.
	severity_value	integer DEFAULT 1	Represents the intensity or severity level of the disease for a particular patient.
*	start_date	date	Represents the date when the disease was first diagnosed or recorded.
	end_date	date	Represents the date when the patient recovered from or no longer has the disease.
Indexes			
🔑	diseased_patient_pkey	Primary Key ON person_id, disease_id	Uniquely identifies each record of a person being diagnosed with a specific disease.
Foreign Key			
↱	diseased_patient_person_id_fkey	person_id ↗ ❑ person	
↱	diseased_patient_disease_id_fkey	disease_id ↗ ❑ disease	
Constraints			
	diseased_patient_severity_value_check	((severity_value >= 1) AND (severity_value <= 10))	Ensures that the severity_value is between 1 and 10.

Table healthcare_provider

Idx	Column Name	Definition	Description
*🔑↰	provider_id	serial	Unique numeric ID for identifying each healthcare provider.
	name	varchar(150)	The full name of the healthcare provider
	specialization	varchar(100)	The area of medical practice or expertise
↱	location_id	integer	Identifies the geographical location of the healthcare provider.
Indexes			
🔑	healthcare_provider_pkey	Primary Key ON provider_id	
Foreign Key			
↱	healthcare_provider_location_id_fkey	location_id ↗ ❑ location	
Referring Foreign Key			
↰	person_provider_id_fkey	provider_id ✓ ❑ person	Links the provider_id to the healthcare_provider table.

Table indication

Idx	Column Name	Definition	Description
*🔑↱	medicine_id	integer	Represents the unique identifier of a medicine.
*🔑↱	disease_id	integer	Represents the unique identifier of a disease.
	indication_date	date	Represents the date on which the indication of the medicine for the disease was established.
	effectiveness_percent	double precision	Represents the percentage of effectiveness of the medicine in treating the disease.
Indexes			
🔑	indication_pkey	Primary Key ON medicine_id, disease_id	
Foreign Key			
↱	indication_medicine_id_fkey	medicine_id ↗ ❑ medicine	Ensures that each medicine_id in the indication table corresponds to a valid medicine in the medicine table.
↱	indication_disease_id_fkey	disease_id ↗ ❑ disease	Ensures that each disease_id in the indication table corresponds to a valid disease in the disease table.
Constraints			
	indication_effectiveness_percent_check	((effectiveness_percent >= (0)::double precision) AND (effectiveness_percent <= (100)::double precision))	Ensures that the effectiveness_percent value is between 0 and 100.

Table insurance

Idx	Column Name	Definition	Description
*🔑↰	insurance_id	serial	A unique numeric identifier for each insurance record.
	provider_name	varchar(150)	The name of the insurance provider or company
	plan_name	varchar(150)	The specific name or title of the insurance plan
	coverage_percent	double precision	Represents the percentage of healthcare costs covered by the insurance plan.
Indexes			
🔑	insurance_pkey	Primary Key ON insurance_id	
Referring Foreign Key			
↰	person_insurance_id_fkey	insurance_id ✓ ❑ person	Links the insurance_id to the insurance table.
Constraints			
	insurance_coverage_percent_check	((coverage_percent >= (0)::double precision) AND (coverage_percent <= (100)::double precision))	Ensures that the coverage_percent value is between 0 and 100.

Table location

Idx	Column Name	Definition	Description
*🔑↰	location_id	serial	Represents the unique ID for each location record.
*	city_name	varchar(100)	The name of the city
	state_province_name	varchar(100)	Name of the state or province
*	country_name	varchar(100)	Name of the country where the location exists
*	developing_flag	char(1)	Indicates whether the location is in a developing country. Accepted values are 'Y' (Yes) or 'N' (No).
	wealth_rank_number	integer	Represents the relative economic ranking of the location. Values range from 1 to 10 (1 being the least wealthy and 10 the most).
Indexes			
🔑	location_pkey	Primary Key ON location_id	
Referring Foreign Key			
↰	healthcare_provider_location_id_fkey	location_id ✓ ❑ healthcare_provider	
↰	person_primary_location_id_fkey	location_id ✓ ❑ person(primary_location_id)	Links the primary_location_id to the location table.
Constraints			
	location_developing_flag_check	(developing_flag = ANY (ARRAY['Y'::bpchar, 'N'::bpchar]))	Ensures that the developing_flag value is either 'Y' or 'N'.
	location_wealth_rank_number_check	((wealth_rank_number >= 1) AND (wealth_rank_number <= 10))	Ensures that wealth_rank_number falls within the range of 1 to 10.

Table medicine

Idx	Column Name	Definition	Description
*🔑↰	medicine_id	serial	Represents the unique identifier for a medicine.
	standard_industry_number	varchar(25)	A unique industry identifier for the medicine, often used for regulatory purposes.
*	name	varchar(250)	The commercial or generic name of the medicine
	company	varchar(150)	The name of the pharmaceutical company that manufactures the medicine
	active_ingredient_name	varchar(150)	The primary active ingredient in the medicine
Indexes			
🔑	medicine_pkey	Primary Key ON medicine_id	
Referring Foreign Key			
↰	indication_medicine_id_fkey	medicine_id ✓ ❑ indication	Ensures that each medicine_id in the indication table corresponds to a valid medicine in the medicine table.

Table person

Idx	Column Name	Definition	Description
*🔑↰	person_id	serial	Represents the unique identifier for a person.
*	last_name	varchar(50)	The surname or family name of the person.
	first_name	varchar(50)	The given name of the person
*	gender	char(1)	Represents the gender of the person.
↱	primary_location_id	integer	Represents the primary location associated with the person.
↱	race_cd	char(5)	Represents the race or ethnicity of the person.
↱	insurance_id	integer	Represents the insurance policy associated with the person.
↱	provider_id	integer	Represents the healthcare provider primarily responsible for the person.
Indexes			
🔑	person_pkey	Primary Key ON person_id	A unique numeric identifier for each person.
Foreign Key			
↱	person_primary_location_id_fkey	primary_location_id ↗ ❑ location(location_id)	Links the primary_location_id to the location table.
↱	person_race_cd_fkey	race_cd ↗ ❑ race(race_code)	Links the race_cd to the race table.
↱	person_insurance_id_fkey	insurance_id ↗ ❑ insurance	Links the insurance_id to the insurance table.
↱	person_provider_id_fkey	provider_id ↗ ❑ healthcare_provider	Links the provider_id to the healthcare_provider table.
Referring Foreign Key			
↰	diseased_patient_person_id_fkey	person_id ↙ ❑ diseased_patient	
↰	test_person_id_fkey	person_id ↙ ❑ test	Links the person_id to the person table. Ensures that each test record is associated with a valid person entry.
Constraints			
	person_gender_check	(gender = ANY (ARRAY['M'::bpchar, 'F'::bpchar, 'U'::bpchar]))	Ensures that the gender value is one of 'M', 'F', or 'U'. If a healthcare system registers a patient whose gender is not specified or recorded, the database will store it as U.

Table race

Idx	Column Name	Definition	Description
*🔑↰	race_code	char(5)	Represents the unique identifier for a race or ethnic group.
*	race_description	varchar(100)	Descriptive name of the race or ethnic group
Indexes			
🔑	race_pkey	Primary Key ON race_code	A unique 5-character code representing a specific race or ethnicity
Referring Foreign Key			
↰	person_race_cd_fkey	race_code ↙ ❑ person(race_cd)	Links the race_cd to the race table.
↰	race_disease_propensity_race_code_fkey	race_code ↙ ❑ race_disease_propensity	Links the race_code to the race table.

Table race_disease_propensity

Idx	Column Name	Definition	Description
*🔑↱	race_code	char(5)	Represents the unique identifier for a race or ethnic group.
*🔑↱	disease_id	integer	Represents the unique identifier for a disease.
	propensity_value	integer	Indicates the likelihood or tendency of a race being affected by a particular disease.
Indexes			
🔑	race_disease_propensity_pkey	Primary Key ON race_code, disease_id	Primary key index on the composite key (race_code, disease_id) to ensure unique pairs.
Foreign Key			
↱	race_disease_propensity_race_code_fkey	race_code ↗ ❑ race	Links the race_code to the race table.
↱	race_disease_propensity_disease_id_fkey	disease_id ↗ ❑ disease	Links the disease_id to the disease table.
Constraints			
	race_disease_propensity_propensity_value_check	((propensity_value >= 1) AND (propensity_value <= 10))	

Table test

Idx	Column Name	Definition	Description
*🔑	test_id	serial	Represents the unique identifier for each medical test performed.
↱	person_id	integer	Represents the identifier of the person who underwent the test.
↱	disease_id	integer	Represents the identifier of the disease for which the test was conducted.
*	test_date	date	Indicates the date when the medical test was performed.
	result	varchar(50)	Contains the textual result of the test
	result_value	double precision	Represents a numerical value associated with the test result
Indexes			
🔑	test_pkey	Primary Key ON test_id	Primary key index on test_id to ensure quick lookups and unique identification of each test record.
Foreign Key			
↱	test_person_id_fkey	person_id ↗ ❑ person	Links the person_id to the person table. Ensures that each test record is associated with a valid person entry.
↱	test_disease_id_fkey	disease_id ↗ ❑ disease	Links the disease_id to the disease table. Ensures that each test record corresponds to a valid disease entry.

Step 3: PostgreSQL OLTP Database

3.1 Table Creation Sql

File Name: Table Creation queries.sql

Query | Query History

```sql
-- 1. Disease Type
CREATE TABLE disease_type (
    disease_type_code CHAR(5) PRIMARY KEY,
    disease_type_description VARCHAR(1000) NOT NULL,
    exclusions_other_note_text VARCHAR(2000)
);

-- 2. Disease
CREATE TABLE disease (
    disease_id SERIAL PRIMARY KEY,
    disease_name VARCHAR(100) NOT NULL,
    intensity_level_qty INT DEFAULT 1 CHECK (intensity_level_qty BETWEEN 1 AND 10),
    disease_type_cd CHAR(5) NOT NULL REFERENCES disease_type(disease_type_code),
    source_disease_cd INT REFERENCES disease(disease_id)
);

-- 3. Race
CREATE TABLE race (
    race_code CHAR(5) PRIMARY KEY,
    race_description VARCHAR(100) NOT NULL
);

-- 4. Location
CREATE TABLE location (
    location_id SERIAL PRIMARY KEY,
    city_name VARCHAR(100) NOT NULL,
    state_province_name VARCHAR(100),
    country_name VARCHAR(100) NOT NULL,
    developing_flag CHAR(1) NOT NULL CHECK (developing_flag IN ('Y', 'N')),
    wealth_rank_number INT CHECK (wealth_rank_number BETWEEN 1 AND 10)
);

-- 5. Insurance
CREATE TABLE insurance (
    insurance_id SERIAL PRIMARY KEY,
    provider_name VARCHAR(150),
    plan_name VARCHAR(150),
    coverage_percent FLOAT CHECK (coverage_percent BETWEEN 0 AND 100)
);
```

Query | Query History

```sql
-- 6. Healthcare Provider
CREATE TABLE healthcare_provider (
    provider_id SERIAL PRIMARY KEY,
    name VARCHAR(150),
    specialization VARCHAR(100),
    location_id INT REFERENCES location(location_id)
);

-- 7. Person
CREATE TABLE person (
    person_id SERIAL PRIMARY KEY,
    last_name VARCHAR(50) NOT NULL,
    first_name VARCHAR(50),
    gender CHAR(1) NOT NULL CHECK (gender IN ('M', 'F', 'U')),
    primary_location_id INT REFERENCES location(location_id),
    race_cd CHAR(5) REFERENCES race(race_code),
    insurance_id INT REFERENCES insurance(insurance_id),
    provider_id INT REFERENCES healthcare_provider(provider_id)
);

-- 8. Diseased Patient
CREATE TABLE diseased_patient (
    person_id INT NOT NULL REFERENCES person(person_id),
    disease_id INT NOT NULL REFERENCES disease(disease_id),
    severity_value INT DEFAULT 1 CHECK (severity_value BETWEEN 1 AND 10),
    start_date DATE NOT NULL,
    end_date DATE,
    PRIMARY KEY (person_id, disease_id)
);

-- 9. Medicine
CREATE TABLE medicine (
    medicine_id SERIAL PRIMARY KEY,
    standard_industry_number VARCHAR(25),
    name VARCHAR(250) NOT NULL,
    company VARCHAR(150),
    active_ingredient_name VARCHAR(150)
);
```

```sql
-- 10. Indication
CREATE TABLE indication (
    medicine_id INT NOT NULL REFERENCES medicine(medicine_id),
    disease_id INT NOT NULL REFERENCES disease(disease_id),
    indication_date DATE,
    effectiveness_percent FLOAT CHECK (effectiveness_percent BETWEEN 0 AND 100),
    PRIMARY KEY (medicine_id, disease_id)
);

-- 11. Race Disease Propensity
CREATE TABLE race_disease_propensity (
    race_code CHAR(5) NOT NULL REFERENCES race(race_code),
    disease_id INT NOT NULL REFERENCES disease(disease_id),
    propensity_value INT CHECK (propensity_value BETWEEN 1 AND 10),
    PRIMARY KEY (race_code, disease_id)
);

-- 12. Test
CREATE TABLE test (
    test_id SERIAL PRIMARY KEY,
    person_id INT REFERENCES person(person_id),
    disease_id INT REFERENCES disease(disease_id),
    test_date DATE NOT NULL,
    result VARCHAR(50),
    result_value FLOAT
);

```

3.2 Insertion into Table

File name: Table insertion queries.sql

```sql
-- 1. Disease Type
INSERT INTO disease_type (disease_type_code, disease_type_description, exclusions_other_note_text) VALUES
('INF', 'Infectious Disease', 'None'),
('CHR', 'Chronic Disease', 'Exclude mental disorders'),
('GEN', 'Genetic Disorder', 'Inherited conditions'),
('NUR', 'Neurological Disorder', 'Exclude trauma-induced'),
('CAN', 'Cancer', 'Malignant only'),
('MNT', 'Mental Health Disorder', 'Psychological conditions'),
('IMM', 'Immune System Disorder', 'Autoimmune types'),
('CAR', 'Cardiovascular Disease', 'Heart-related conditions'),
('RES', 'Respiratory Disease', 'Exclude allergy only'),
('END', 'Endocrine Disorder', 'Hormonal issues');

-- 2. Disease
INSERT INTO disease (disease_name, intensity_level_qty, disease_type_cd, source_disease_cd) VALUES
('COVID-19', 8, 'INF', NULL),
('Diabetes', 5, 'CHR', NULL),
('Tuberculosis', 7, 'INF', NULL),
('Breast Cancer', 9, 'CAN', NULL),
('Depression', 6, 'MNT', NULL),
('Asthma', 5, 'RES', NULL),
('Lupus', 7, 'IMM', NULL),
('Heart Attack', 9, 'CAR', NULL),
('Parkinson''s', 6, 'NUR', NULL),
('Thyroid Disorder', 4, 'END', NULL);

-- 3. Race
INSERT INTO race (race_code, race_description) VALUES
('ASN', 'Asian'),
('BLK', 'Black or African American'),
('WHT', 'White'),
('LAT', 'Latino/Hispanic'),
('NAT', 'Native American'),
('PAC', 'Pacific Islander'),
('MID', 'Middle Eastern'),
('MUL', 'Multiracial'),
('OTH', 'Other'),
('UND', 'Undisclosed');

```

Query | Query History

```sql
-- 4. Location
INSERT INTO location(city_name, state_province_name, country_name, developing_flag, wealth_rank_number) VALUES
('New York', 'NY', 'USA', 'N', 9),
('Lalitpur', 'Bagmati', 'Nepal', 'Y', 5),
('London', 'England', 'UK', 'N', 10),
('Delhi', 'Delhi', 'India', 'Y', 6),
('Tokyo', 'Tokyo', 'Japan', 'N', 10),
('Lagos', 'Lagos', 'Nigeria', 'Y', 3),
('Berlin', 'Berlin', 'Germany', 'N', 9),
('Rio', 'RJ', 'Brazil', 'Y', 6),
('Toronto', 'ON', 'Canada', 'N', 9),
('Sydney', 'NSW', 'Australia', 'N', 10);


-- 5. Insurance
INSERT INTO insurance (provider_name, plan_name, coverage_percent) VALUES
('Aetna', 'Silver Plan', 80),
('Blue Cross', 'Gold Plan', 90),
('United Health', 'Bronze Plan', 70),
('Cigna', 'Platinum Plan', 95),
('Humana', 'Basic Care', 75),
('Kaiser', 'Family Health', 85),
('Anthem', 'Essential Health', 88),
('WellCare', 'WellBasic', 73),
('Oscar', 'Core Plan', 78),
('Molina', 'Complete Care', 92);

-- 6. Healthcare Provider
INSERT INTO healthcare_provider (name, specialization, location_id) VALUES
('Dr. Smith', 'Internal Medicine', 1),
('Dr. Sharma', 'Pulmonologist', 2),
('Dr. Ahmed', 'Cardiologist', 3),
('Dr. Lee', 'Neurologist', 4),
('Dr. Mehta', 'Oncologist', 5),
('Dr. Chen', 'Endocrinologist', 6),
('Dr. Patel', 'General Practitioner', 7),
('Dr. Gomez', 'Psychiatrist', 8),
('Dr. Kim', 'Immunologist', 9),
('Dr. Singh', 'Family Medicine', 10);

```

Query | Query History

```sql
-- 7. Person
INSERT INTO person (last_name, first_name, gender, primary_location_id, race_cd, insurance_id, provider_id) VALUES
('Lee', 'Min', 'F', 1, 'ASN', 1, 1),
('Jackson', 'Tina', 'F', 1, 'BLK', 2, 1),
('Bhandari', 'Raj', 'M', 2, 'ASN', 3, 2),
('Smith', 'John', 'M', 3, 'WHT', 4, 3),
('Khan', 'Sara', 'F', 4, 'MID', 5, 4),
('Perez', 'Luis', 'M', 5, 'LAT', 6, 5),
('Whitecloud', 'Maya', 'F', 6, 'NAT', 7, 6),
('Park', 'Jin', 'M', 7, 'PAC', 8, 7),
('Nguyen', 'Linh', 'F', 8, 'ASN', 9, 8),
('Brown', 'Alicia', 'F', 9, 'BLK', 10, 9);

-- 8. Medicine
INSERT INTO medicine (standard_industry_number, name, company, active_ingredient_name) VALUES
('SIN123', 'Remdesivir', 'Gilead', 'GS-5734'),
('SIN456', 'Metformin', 'Sun Pharma', 'Metformin Hydrochloride'),
('SIN789', 'Aspirin', 'Bayer', 'Acetylsalicylic Acid'),
('SIN101', 'Lipitor', 'Pfizer', 'Atorvastatin'),
('SIN202', 'Prozac', 'Eli Lilly', 'Fluoxetine'),
('SIN303', 'Prednisone', 'Teva', 'Prednisone'),
('SIN404', 'Lisinopril', 'Novartis', 'Lisinopril'),
('SIN505', 'Tamoxifen', 'AstraZeneca', 'Tamoxifen Citrate'),
('SIN606', 'Levothyroxine', 'AbbVie', 'Levothyroxine'),
('SIN707', 'Insulin', 'Novo Nordisk', 'Insulin Human');

-- 9. Indication
INSERT INTO indication (medicine_id, disease_id, indication_date, effectiveness_percent) VALUES
(1, 1, '2020-06-01', 65.0),
(2, 2, '2018-01-15', 78.5),
(3, 8, '2021-02-01', 84.2),
(4, 8, '2020-03-21', 89.0),
(5, 5, '2022-05-10', 60.0),
(6, 7, '2021-07-14', 74.6),
(7, 8, '2019-09-30', 77.5),
(8, 4, '2021-11-01', 81.1),
(9, 10, '2022-03-10', 68.0),
(10, 2, '2017-10-05', 88.9);


```

```sql
-- 10. Diseased Patient
INSERT INTO diseased_patient (person_id, disease_id, severity_value, start_date, end_date) VALUES
(1, 1, 7, '2020-05-15', '2020-06-30'),
(2, 2, 6, '2019-03-10', NULL),
(3, 3, 8, '2021-11-05', NULL),
(4, 4, 9, '2022-02-01', NULL),
(5, 5, 5, '2021-01-20', NULL),
(6, 6, 6, '2020-12-10', NULL),
(7, 7, 7, '2021-03-15', NULL),
(8, 8, 8, '2022-07-07', NULL),
(9, 9, 6, '2023-05-01', NULL),
(10, 10, 5, '2021-09-25', NULL);

-- 11. Race Disease Propensity
INSERT INTO race_disease_propensity (race_code, disease_id, propensity_value) VALUES
('ASN', 2, 7),
('BLK', 2, 8),
('WHT', 1, 5),
('LAT', 5, 6),
('NAT', 4, 7),
('PAC', 3, 6),
('MID', 8, 8),
('MUL', 6, 7),
('OTH', 7, 5),
('UND', 9, 6);

-- 12. Test
INSERT INTO test (person_id, disease_id, test_date, result, result_value) VALUES
(1, 1, '2020-05-10', 'Positive', 1.0),
(2, 2, '2021-03-05', 'Negative', 0.0),
(3, 3, '2021-11-04', 'Positive', 1.0),
(4, 4, '2022-02-10', 'Positive', 1.0),
(5, 5, '2021-01-25', 'Negative', 0.0),
(6, 6, '2020-12-15', 'Positive', 1.0),
(7, 7, '2021-03-20', 'Negative', 0.0),
(8, 8, '2022-07-10', 'Positive', 1.0),
(9, 9, '2023-05-03', 'Negative', 0.0),
(10, 10, '2021-09-27', 'Positive', 1.0);

```

Step 4: Run Operational Queries & DML Operations

4.1 Operational Queries (Reports)

File name: operationalquery.sql

Query | Query History

```sql
-- 1. List all patients and the diseases they have
SELECT p.first_name, p.last_name, d.disease_name, dp.severity_value
FROM diseased_patient dp
JOIN person p ON dp.person_id = p.person_id
JOIN disease d ON dp.disease_id = d.disease_id;
```

Data Output | Messages | Notifications

	first_name character varying (50)	last_name character varying (50)	disease_name character varying (100)	severity_value integer
1	Min	Lee	COVID-19	7
2	Tina	Jackson	Diabetes	6
3	Raj	Bhandari	Tuberculosis	8
4	John	Smith	Breast Cancer	9
5	Sara	Khan	Depression	5
6	Luis	Perez	Asthma	6
7	Maya	Whitecloud	Lupus	7
8	Jin	Park	Heart Attack	8
9	Linh	Nguyen	Parkinson's	6
10	Alicia	Brown	Thyroid Disorder	5

```sql
-- 2. Find the most common diseases by race
SELECT r.race_description, d.disease_name, COUNT(*) AS case_count
FROM diseased_patient dp
JOIN person p ON dp.person_id = p.person_id
JOIN disease d ON dp.disease_id = d.disease_id
JOIN race r ON p.race_cd = r.race_code
GROUP BY r.race_description, d.disease_name
ORDER BY case_count DESC;
```

Data Output | Messages | Notifications

	race_description character varying (100)	disease_name character varying (100)	case_count bigint
1	Native American	Lupus	1
2	Asian	COVID-19	1
3	White	Breast Cancer	1
4	Black or African American	Diabetes	1
5	Middle Eastern	Depression	1
6	Asian	Tuberculosis	1
7	Asian	Parkinson's	1
8	Pacific Islander	Heart Attack	1
9	Black or African American	Thyroid Disorder	1
10	Latino/Hispanic	Asthma	1

```sql
-- 3. Get medicine effectiveness for each disease
SELECT d.disease_name, m.name AS medicine_name, i.effectiveness_percent
FROM indication i
JOIN disease d ON i.disease_id = d.disease_id
JOIN medicine m ON i.medicine_id = m.medicine_id;
```

Data Output | Messages | Notifications

	disease_name character varying (100)	medicine_name character varying (250)	effectiveness_percent double precision
1	COVID-19	Remdesivir	65
2	Diabetes	Metformin	78.5
3	Heart Attack	Aspirin	84.2
4	Heart Attack	Lipitor	89
5	Depression	Prozac	60
6	Lupus	Prednisone	74.6
7	Heart Attack	Lisinopril	77.5
8	Breast Cancer	Tamoxifen	81.1
9	Thyroid Disorder	Levothyroxine	68
10	Diabetes	Insulin	88.9

```sql
-- 4. Find patients who tested positive for any disease
SELECT p.first_name, p.last_name, d.disease_name, t.test_date
FROM test t
JOIN person p ON t.person_id = p.person_id
JOIN disease d ON t.disease_id = d.disease_id
WHERE t.result = 'Positive';
```

Data Output | Messages | Notifications

	first_name character varying (50)	last_name character varying (50)	disease_name character varying (100)	test_date date
1	Min	Lee	COVID-19	2020-05-10
2	Raj	Bhandari	Tuberculosis	2021-11-04
3	John	Smith	Breast Cancer	2022-02-10
4	Luis	Perez	Asthma	2020-12-15
5	Jin	Park	Heart Attack	2022-07-10
6	Alicia	Brown	Thyroid Disorder	2021-09-27

```sql
-- 5. Count of active disease cases by location
SELECT l.city_name, l.country_name, COUNT(*) AS active_cases
FROM diseased_patient dp
JOIN person p ON dp.person_id = p.person_id
JOIN location l ON p.primary_location_id = l.location_id
WHERE dp.end_date IS NULL
GROUP BY l.city_name, l.country_name
ORDER BY active_cases DESC;
```

Data Output | Messages | Notifications

	city_name character varying (100)	country_name character varying (100)	active_cases bigint
1	Berlin	Germany	1
2	Delhi	India	1
3	Lagos	Nigeria	1
4	Lalitpur	Nepal	1
5	London	UK	1
6	New York	USA	1
7	Rio	Brazil	1
8	Tokyo	Japan	1
9	Toronto	Canada	1

4.2 DML Operations (Insert, Update & Delete)

File name: dmloperation.sql

Query | Query History

```sql
-- Insert a new test for an existing patient
INSERT INTO test (person_id, disease_id, test_date, result, result_value)
VALUES (1, 2, '2024-01-01', 'Negative', 0.0);
```

Data Output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 104 msec.
```

```sql
-- Update severity of a disease case
UPDATE diseased_patient
SET severity_value = 9
WHERE person_id = 2 AND disease_id = 2;
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 89 msec.
```

```sql
-- Try deleting a medicine (will fail if it is referenced in indication table)
DELETE FROM medicine
WHERE medicine_id = 1;
```

Data Output | Messages | Notifications

```
ERROR:  Key (medicine_id)=(1) is still referenced from table "indication".update or delete on table "medicine" violates foreign key constraint "indication_medicine_id_fkey" on table "indication"

ERROR:  update or delete on table "medicine" violates foreign key constraint "indication_medicine_id_fkey" on table "indication"
SQL state: 23503
Detail: Key (medicine_id)=(1) is still referenced from table "indication".
```

```sql
-- Delete test data for a patient (if allowed)
DELETE FROM test
WHERE person_id = 3;
```

Data Output | Messages | Notifications

```
DELETE 1

Query returned successfully in 86 msec.
```

Step 5: Dimensional Model (Star Schema)

5.1 Creation of dimension tables

File name: dimcreation.sql

```sql

-- Create schema for Data Warehouse
CREATE SCHEMA IF NOT EXISTS disease_dwh;
SET search_path TO disease_dwh;

-- 1. dim_patient
CREATE TABLE dim_patient (
    patient_id SERIAL PRIMARY KEY,
    person_id INT,
    full_name VARCHAR(100),
    gender CHAR(1),
    birth_year INT,
    race_code CHAR(5),
    location_id INT
);

-- 2. dim_disease
CREATE TABLE dim_disease (
    disease_id INT PRIMARY KEY,
    disease_name VARCHAR(100),
    intensity_level INT,
    disease_type VARCHAR(100)
);

-- 3. dim_medicine
CREATE TABLE dim_medicine (
    medicine_id INT PRIMARY KEY,
    name VARCHAR(250),
    company VARCHAR(150),
    active_ingredient_name VARCHAR(150)
);

```

Query | Query History

```sql
-- 4. dim_location
CREATE TABLE dim_location (
    location_id INT PRIMARY KEY,
    city_name VARCHAR(100),
    state_province_name VARCHAR(100),
    country_name VARCHAR(100),
    wealth_rank_number INT,
    developing_flag CHAR(1)
);

-- 5. dim_race
CREATE TABLE dim_race (
    race_code CHAR(5) PRIMARY KEY,
    race_description VARCHAR(100)
);

-- 6. dim_insurance
CREATE TABLE dim_insurance (
    insurance_id INT PRIMARY KEY,
    provider_name VARCHAR(150),
    plan_name VARCHAR(150),
    coverage_percent FLOAT
);

-- 7. dim_provider
CREATE TABLE dim_provider (
    provider_id INT PRIMARY KEY,
    name VARCHAR(150),
    specialization VARCHAR(100),
    location_id INT
);

```

```sql
-- 8. dim_date
CREATE TABLE disease_dwh.dim_date (
    date_id DATE PRIMARY KEY,
    day INT,
    month INT,
    year INT,
    quarter INT,
    week INT,
    is_weekend BOOLEAN
);

-- 9. fact_disease_event
CREATE TABLE disease_dwh.fact_disease_event (
    fact_id SERIAL PRIMARY KEY,
    person_id INT,
    disease_id INT,
    medicine_id INT,
    location_id INT,
    race_code CHAR(5),
    insurance_id INT,
    provider_id INT,
    test_result VARCHAR(50),
    test_value FLOAT,
    severity INT,
    start_date DATE,
    end_date DATE,
    date_id DATE REFERENCES disease_dwh.dim_date(date_id) -- FK to dim_date
);

```

5.2 Population of Dimension Tables

File name: dimensiontablepopulate.sql

```sql

-- ETL Script: Populate Dimension Tables

-- 1. dim_patient
INSERT INTO disease_dwh.dim_patient (person_id, full_name, gender, birth_year, race_code, location_id)
SELECT
    person_id,
    CONCAT(first_name, ' ', last_name),
    gender,
    EXTRACT(YEAR FROM CURRENT_DATE) - 30,  -- Assume everyone is 30 years old for mock data
    race_cd,
    primary_location_id
FROM person;

-- 2. dim_disease
INSERT INTO disease_dwh.dim_disease (disease_id, disease_name, intensity_level, disease_type)
SELECT
    d.disease_id,
    d.disease_name,
    d.intensity_level_qty,
    dt.disease_type_description
FROM disease d
JOIN disease_type dt ON d.disease_type_cd = dt.disease_type_code;

-- 3. dim_medicine
INSERT INTO disease_dwh.dim_medicine (medicine_id, name, company, active_ingredient_name)
SELECT medicine_id, name, company, active_ingredient_name
FROM medicine;

-- 4. dim_location
INSERT INTO disease_dwh.dim_location (location_id, city_name, state_province_name, country_name, wealth_rank_number, developing_flag)
SELECT location_id, city_name, state_province_name, country_name, wealth_rank_number, developing_flag
FROM location;

-- 5. dim_race
INSERT INTO disease_dwh.dim_race (race_code, race_description)
SELECT race_code, race_description
FROM race;

```

```sql
-- 6. dim_insurance
INSERT INTO disease_dwh.dim_insurance (insurance_id, provider_name, plan_name, coverage_percent)
SELECT insurance_id, provider_name, plan_name, coverage_percent
FROM insurance;

-- 7. dim_provider
INSERT INTO disease_dwh.dim_provider (provider_id, name, specialization, location_id)
SELECT provider_id, name, specialization, location_id
FROM healthcare_provider;

-- Generate full date range in dim_date (2020-01-01 to 2025-12-31)
-- Extend dim_date to start from 2015
INSERT INTO disease_dwh.dim_date (date_id, day, month, year, quarter, week, is_weekend)
SELECT
    d::DATE,
    EXTRACT(DAY FROM d),
    EXTRACT(MONTH FROM d),
    EXTRACT(YEAR FROM d),
    EXTRACT(QUARTER FROM d),
    EXTRACT(WEEK FROM d),
    CASE WHEN EXTRACT(DOW FROM d) IN (0, 6) THEN TRUE ELSE FALSE END
FROM generate_series('2015-01-01'::DATE, '2019-12-31'::DATE, '1 day'::INTERVAL) d;


SELECT COUNT(*) FROM disease_dwh.dim_date;

```

```sql
-- 8. fact_disease_event
INSERT INTO disease_dwh.fact_disease_event (
    person_id, disease_id, medicine_id, location_id, race_code,
    insurance_id, provider_id, test_result, test_value, severity,
    start_date, end_date, date_id
)
SELECT
    p.person_id,
    dp.disease_id,
    i.medicine_id,
    p.primary_location_id,
    p.race_cd,
    p.insurance_id,
    p.provider_id,
    t.result,
    t.result_value,
    dp.severity_value,
    dp.start_date,
    dp.end_date,
    dp.start_date
FROM diseased_patient dp
JOIN person p ON dp.person_id = p.person_id
LEFT JOIN test t ON t.person_id = dp.person_id AND t.disease_id = dp.disease_id
LEFT JOIN indication i ON i.disease_id = dp.disease_id;

```

SCD 2

```sql
DROP TABLE IF EXISTS disease_dwh.dim_patient;

CREATE TABLE disease_dwh.dim_patient (
    patient_sk SERIAL PRIMARY KEY,              -- Surrogate key
    person_id INT,                              -- Business key
    full_name VARCHAR(100),
    gender CHAR(1),
    birth_year INT,
    race_code CHAR(5),
    location_id INT,
    insurance_id INT,
    effective_start_date DATE,                  -- SCD2 tracking
    effective_end_date DATE,
    is_current BOOLEAN
);

SELECT * FROM disease_dwh.dim_patient;
```

Data Output | Messages | Notifications

patient_sk [PK] integer	person_id integer	full_name character varying (100)	gender character (1)	birth_year integer	race_code character (5)	location_id integer	insurance_id integer	effective_start_date date	effective_end_date date	is_current boolean

```sql
INSERT INTO disease_dwh.dim_patient (person_id, full_name, gender, birth_year, race_code, location_id)
SELECT
    person_id,
    CONCAT(first_name, ' ', last_name),
    gender,
    EXTRACT(YEAR FROM CURRENT_DATE) - 30,  -- Assume everyone is 30 years old for mock data
    race_cd,
    primary_location_id
FROM person;

Select * from disease_dwh.dim_patient;

```

Data Output | Messages | Notifications

Showing rows: 1 to 10 | Page No: 1 of

	patient_sk [PK] integer	person_id integer	full_name character varying (100)	gender character (1)	birth_year integer	race_code character (5)	location_id integer	insurance_id integer	effective_start_date date	effective_end_date date	is_current boolean
1	1	1	Min Lee	F	1995	ASN	1	[null]	[null]	[null]	[null]
2	2	2	Tina Jackson	F	1995	BLK	1	[null]	[null]	[null]	[null]
3	3	3	Raj Bhandari	M	1995	ASN	2	[null]	[null]	[null]	[null]
4	4	4	John Smith	M	1995	WHT	3	[null]	[null]	[null]	[null]
5	5	5	Sara Khan	F	1995	MID	4	[null]	[null]	[null]	[null]
6	6	6	Luis Perez	M	1995	LAT	5	[null]	[null]	[null]	[null]
7	7	7	Maya Whitecloud	F	1995	NAT	6	[null]	[null]	[null]	[null]
8	8	8	Jin Park	M	1995	PAC	7	[null]	[null]	[null]	[null]
9	9	9	Linh Nguyen	F	1995	ASN	8	[null]	[null]	[null]	[null]
10	10	10	Alicia Brown	F	1995	BLK	9	[null]	[null]	[null]	[null]

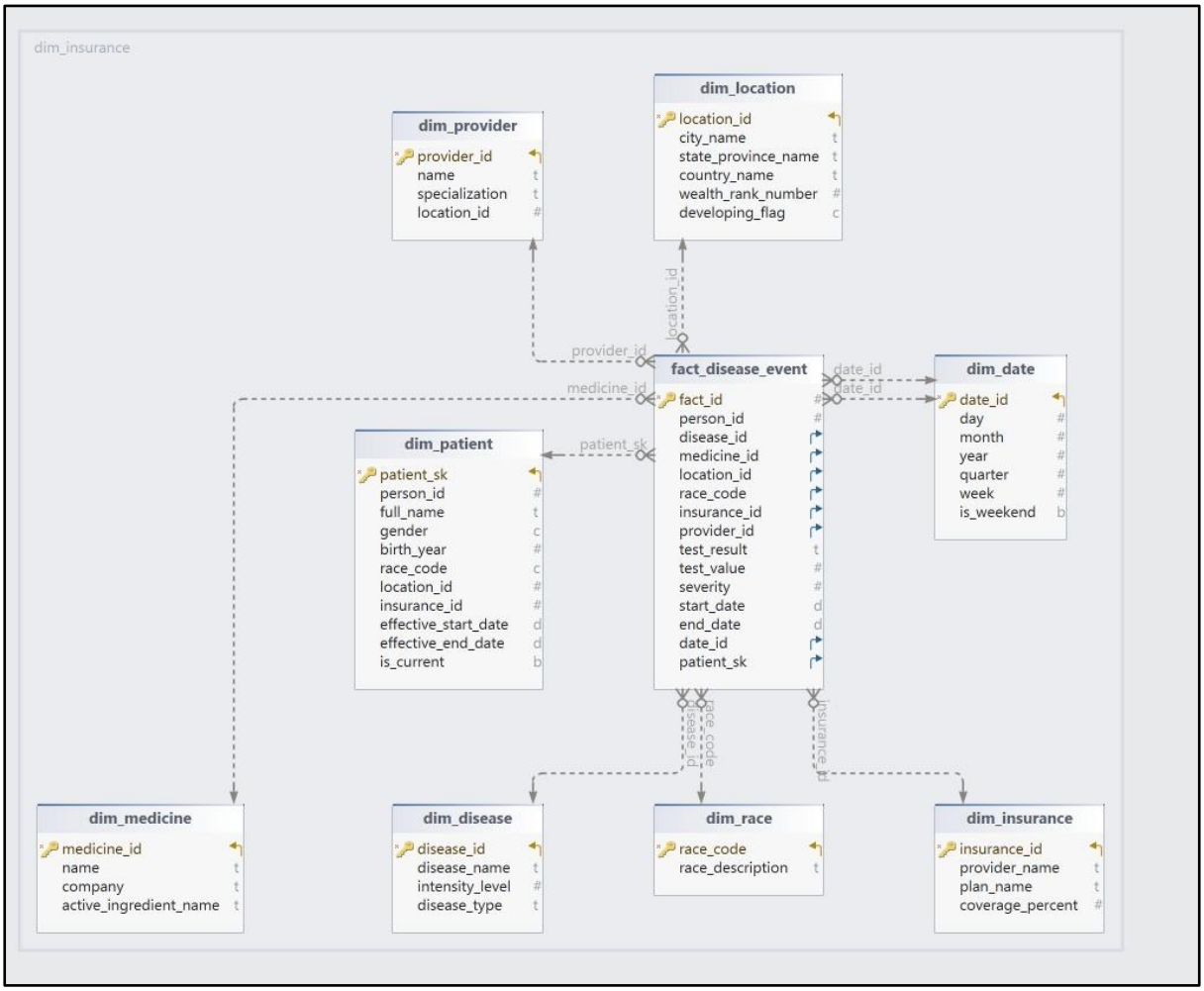
```sql
INSERT INTO disease_dwh.dim_patient (
    person_id, full_name, gender, birth_year, race_code, location_id,
    insurance_id, effective_start_date, effective_end_date, is_current
)
VALUES (
    3, 'Raj Bhandari', 'M', 1995, 'ASN', 5, 5,
    CURRENT_DATE, NULL, TRUE
);

Select * from disease_dwh.dim_patient;
```

Data Output | Messages | Notifications

Showing rows: 1 to 11 | Page No: 1

	patient_sk [PK] integer	person_id integer	full_name character varying (100)	gender character (1)	birth_year integer	race_code character (5)	location_id integer	insurance_id integer	effective_start_date date	effective_end_date date	is_current boolean
1	1	1	Min Lee	F	1995	ASN	1	[null]	[null]	[null]	[null]
2	2	2	Tina Jackson	F	1995	BLK	1	[null]	[null]	[null]	[null]
3	4	4	John Smith	M	1995	WHT	3	[null]	[null]	[null]	[null]
4	5	5	Sara Khan	F	1995	MID	4	[null]	[null]	[null]	[null]
5	6	6	Luis Perez	M	1995	LAT	5	[null]	[null]	[null]	[null]
6	7	7	Maya Whitecloud	F	1995	NAT	6	[null]	[null]	[null]	[null]
7	8	8	Jin Park	M	1995	PAC	7	[null]	[null]	[null]	[null]
8	9	9	Linh Nguyen	F	1995	ASN	8	[null]	[null]	[null]	[null]
9	10	10	Alicia Brown	F	1995	BLK	9	[null]	[null]	[null]	[null]
10	3	3	Raj Bhandari	M	1995	ASN	2	[null]	[null]	2025-05-01	false
11	11	3	Raj Bhandari	M	1995	ASN	5	5	2025-05-02	[null]	true

5.3 Dimensional Modeling



This dimensional model represents the Disease Data Management and Analytics System in a data warehouse context. It uses a star schema structure, where the central fact table is

surrounded by dimension tables. This schema is designed to facilitate analytical queries and reporting related to healthcare data.

1. Fact Table: fact_disease_event

The fact table at the center stores the measurable data related to disease events. It includes the following key attributes:

- fact_id: Unique identifier for each event.
- person_id: ID linking to the patient.
- disease_id: ID referring to the disease diagnosed.
- medicine_id: ID of the prescribed medicine.
- location_id: Location where the event took place.
- insurance_id: Insurance coverage associated with the event.
- provider_id: Healthcare provider responsible.
- test_result: Outcome of a medical test (e.g., positive, negative).
- test_value: Numeric value representing test intensity or result.
- severity: Indicates the intensity level of the disease.
- start_date and end_date: Duration of the event or treatment.
- date_id: Foreign key to the date dimension.
- patient_sk: Surrogate key linking to the patient dimension.

Purpose:

The fact table consolidates various attributes from multiple dimensions to record medical events, allowing analysis on:

- Disease prevalence
- Treatment outcomes
- Regional distribution of diseases
- Healthcare provider performance

2. Dimension Tables:

a. dim_patient

This table captures patient-specific information:

- patient_sk: Surrogate key.
- person_id: Unique identifier.

- full_name, gender, birth_year: Demographic details.
- race_code: Categorical data representing ethnicity.
- location_id: Reference to geographical data.
- insurance_id: Insurance coverage details.
- effective_start_date, effective_end_date: Validity of patient records.
- is_current: Indicates whether the record is active.

b. dim_disease

Describes disease-related information:

- disease_id: Primary key.
- disease_name: Name of the disease.
- intensity_level: Indicates the severity (1 to 10 scale).
- disease_type: Categorizes the disease.

c. dim_provider

Contains healthcare provider data:

- provider_id: Unique identifier.
- name: Provider's name.
- specialization: Area of expertise.
- location_id: Location of the provider.

d. dim_location

Captures geographic details:

- location_id: Unique identifier.
- city_name, state_province_name, country_name: Location hierarchy.
- wealth_rank_number: Socioeconomic status indicator.
- developing_flag: Indicates whether the region is classified as developing.

e. dim_date

Tracks date-related data:

- date_id: Unique identifier.
- day, month, year, quarter, week: Time components.
- is_weekend: Boolean indicator for weekends.

f. dim_medicine

Information about medicines:

- medicine_id: Primary key.
- name: Medicine name.
- company: Manufacturer.
- active_ingredient_name: Main ingredient.

g. dim_race

Stores race/ethnicity descriptions:

- race_code: Unique identifier.
- race_description: Descriptive name of the race.

h. dim_insurance

Contains insurance plan details:

- insurance_id: Unique key.
- provider_name: Name of the insurance provider.
- plan_name: Insurance plan type.
- coverage_percent: Coverage ratio for medical expenses.

Key Features of the Model:

1. Star Schema Design:
    - The central fact table is linked to multiple dimension tables.
    - Ensures efficient querying and faster aggregation.
2. Historical Data Tracking (SCD2):
    - The dim_patient table uses effective_start_date, effective_end_date, and is_current to track patient data changes over time.
3. Data Integration:
    - Integrates geographic, demographic, medical, insurance, and time-related data for comprehensive analysis.
4. Facilitates Advanced Analytics:
    - Supports complex queries, including disease trend analysis, patient demographic insights, provider performance tracking, and geographic disease mapping.

Usage Scenarios for Healthcare Providers:

- Patient Demographics Analysis: Understanding the distribution of diseases based on gender, race, and age.
- Healthcare Service Efficiency: Evaluating the performance of healthcare providers based on treatment outcomes.
- Regional Disease Mapping: Identifying hotspots and trends related to specific diseases.
- Insurance Plan Effectiveness: Analyzing the correlation between insurance plans and treatment

Step 6: Analytical Queries on Star Schema

File name: analyticalqueriesstarschema.sql

Sample Analytical Queries (Star Schema)

```sql
-- Sample Analytical Queries (Star Schema)
--- 1.  Top 5 most common diseases by Case Count
SELECT d.disease_name, COUNT(*) AS case_count
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_disease d ON f.disease_id = d.disease_id
GROUP BY d.disease_name
ORDER BY case_count DESC
LIMIT 5;
```

Data Output | Messages | Notifications

	disease_name character varying (100)	case_count bigint
1	Heart Attack	3
2	Diabetes	2
3	Parkinson's	1
4	Tuberculosis	1
5	Breast Cancer	1

```sql
--- 2. Average Severity of Diseases by Year
SELECT dd.year, d.disease_name, ROUND(AVG(f.severity), 2) AS avg_severity
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_date dd ON f.date_id = dd.date_id
JOIN disease_dwh.dim_disease d ON f.disease_id = d.disease_id
GROUP BY dd.year, d.disease_name
ORDER BY dd.year, avg_severity DESC;
```

Data Output | Messages | Notifications

	year integer	disease_name character varying (100)	avg_severity numeric
1	2019	Diabetes	9.00
2	2020	COVID-19	7.00
3	2020	Asthma	6.00
4	2021	Tuberculosis	8.00
5	2021	Lupus	7.00
6	2021	Depression	5.00
7	2021	Thyroid Disorder	5.00
8	2022	Breast Cancer	9.00
9	2022	Heart Attack	8.00
10	2023	Parkinson's	6.00

```sql
---3. Positive Test Rates by Race
SELECT r.race_description, COUNT(*) FILTER (WHERE f.test_result = 'Positive')::float / COUNT(*) AS positive_rate
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_race r ON f.race_code = r.race_code
GROUP BY r.race_description
ORDER BY positive_rate DESC;
```

Data Output | Messages | Notifications

Showing rows: 1 to 7

	race_description character varying (100)	positive_rate double precision
1	Latino/Hispanic	1
2	Pacific Islander	1
3	White	1
4	Asian	0.3333333333333333
5	Black or African American	0.3333333333333333
6	Middle Eastern	0
7	Native American	0

```sql
---4. Disease Count by Insurance plan
SELECT i.plan_name, COUNT(*) AS total_cases
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_insurance i ON f.insurance_id = i.insurance_id
GROUP BY i.plan_name
ORDER BY total_cases DESC;
```

Data Output | Messages | Notifications

	plan_name character varying (150)	total_cases bigint
1	WellBasic	3
2	Gold Plan	2
3	Core Plan	1
4	Basic Care	1
5	Family Health	1
6	Platinum Plan	1
7	Complete Care	1
8	Essential Health	1
9	Bronze Plan	1
10	Silver Plan	1

```sql
---5. Monthly Case Trends
SELECT dd.year, dd.month, COUNT(*) AS monthly_cases
FROM disease_dwh.fact_disease_event f
JOIN disease_dwh.dim_date dd ON f.date_id = dd.date_id
GROUP BY dd.year, dd.month
ORDER BY dd.year, dd.month;
```

Data Output | Messages | Notifications

	year integer	month integer	monthly_cases bigint
1	2019	3	2
2	2020	5	1
3	2020	12	1
4	2021	1	1
5	2021	3	1
6	2021	9	1
7	2021	11	1
8	2022	2	1
9	2022	7	3
10	2023	5	1

Step 7: NoSQL Comparison Summary

While the disease data model has been implemented in a relational format using PostgreSQL, exploring how this model would be structured in a NoSQL environment reveals important trade-offs and opportunities. This section compares the relational implementation with MongoDB (a document database) and Neo4j (a graph database) to evaluate how these alternative models would handle disease-related data.

A. MongoDB – Document-Oriented NoSQL

MongoDB stores data in collections of documents, similar to JSON objects. Rather than normalizing data across multiple related tables (as in a relational model), MongoDB encourages embedding related data within documents.

Structural Differences

In the relational model:

- person, disease, test, and location are separate tables
- Relationships are enforced through foreign keys

In MongoDB:

- A person document might embed everything related to that person: demographics, location, insurance, test results, and diseases.

Example Document:

```
{
  "person_id": 1,
  "name": "Min Lee",
  "gender": "F",
  "location": {
    "city": "New York",
    "country": "USA"
  },
  "insurance": {
    "provider_name": "Aetna",
    "plan_name": "Silver Plan"
  },
  "diseases": [
    {
      "disease_name": "COVID-19",
      "severity": 7,
      "start_date": "2020-05-15",
      "test": {
        "result": "Positive",
        "value": 1.0,
        "test_date": "2020-05-10"
      }
    }
  ]
}
```

Benefits of MongoDB:

- Simpler and faster queries for retrieving a person's full medical history
- Schema can evolve over time (no need to ALTER TABLE)
- Ideal for real-time applications and REST APIs (e.g., a mobile disease tracker)

Limitations:

- Data duplication (e.g., the same disease might be stored repeatedly across documents)
- No referential integrity where inconsistencies may arise if embedded documents are not synced
- Difficult to perform complex analytical queries like grouping by race or comparing across locations
- Joins are possible using $lookup, but inefficient

Conclusion:

MongoDB is great for individual-centric systems like patient portals, mobile health trackers, or simple health history apps. However, for public health analytics, trend analysis, and BI dashboards, a relational model is more appropriate.

B. Neo4j – Graph Database

Neo4j represents data as nodes (entities) and relationships (edges), making it well-suited for highly connected datasets like disease transmission, referrals, and family histories.

Graph Model Example:

```
(:Person {name: 'Raj'})
   -[:HAS_DISEASE {severity: 6}]-> (:Disease {name: 'Diabetes'})
   -[:LIVES_IN]-> (:Location {city: 'Kathmandu'})
   -[:COVERED_BY]-> (:Insurance {plan_name: 'Gold Plan'})
   -[:VISITED]-> (:Provider {name: 'Dr. Sharma'})
   -[:TESTED_FOR]-> (:Test {result: 'Positive'})
```

Benefits of Neo4j:

- Excellent for tracking relationships such as:
    - "Who did a patient come in contact with before testing positive?"
    - "Which provider treats the most patients with asthma?"
- Pattern-matching with Cypher makes querying networks (e.g., outbreaks) intuitive
- Ideal for epidemiological models, contact tracing, or tracing genetic disease lines

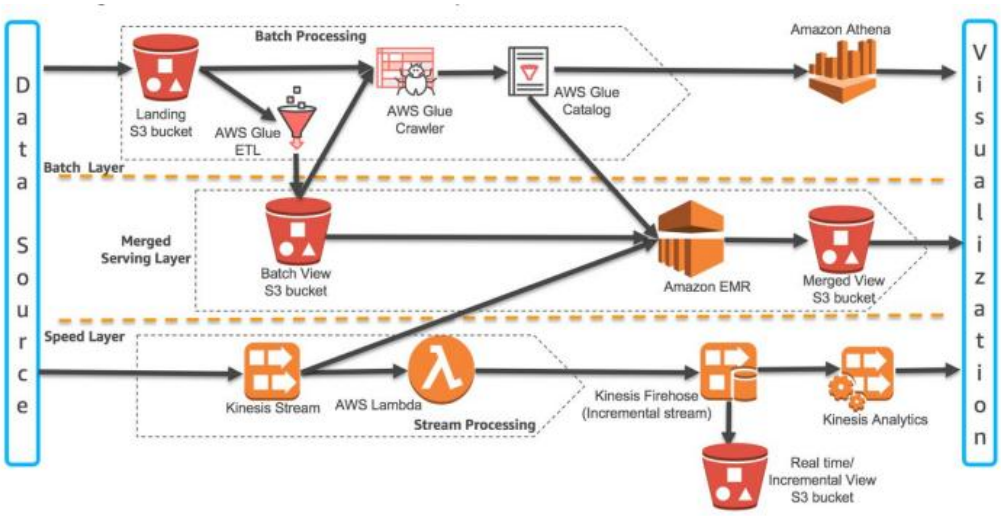
Limitations:

- Less suitable for tabular analytics
- Requires learning Cypher (a new query language)
- Smaller ecosystem for BI tools and ETL support compared to SQL or MongoDB

Overall Conclusion of this step:

While NoSQL systems offer flexibility and performance advantages for specific scenarios, the relational model remains the most robust and scalable choice for structured analytics and data warehousing. However, integrating MongoDB or Neo4j could enhance specific modules such as building a real-time contact tracing app or storing flexible medical notes alongside PostgreSQL analytics stack.

Step 8 AWS Lamda Architecture & Deployment Plan



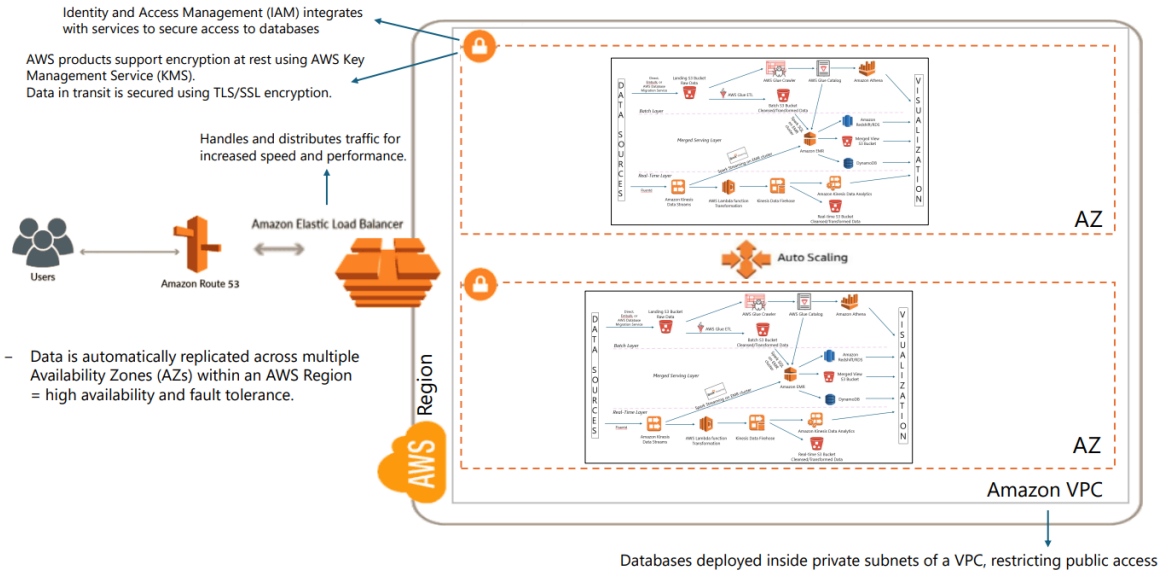
This diagram illustrates how a Lambda Architecture framework in AWS integrates both batch and real-time data streams into a unified data pipeline, which is highly applicable to your disease data project.

At the top of the architecture is the Batch Layer, which processes high-volume, historical data such as multi-year patient records, test results, and hospital reports. The batch data initially lands in an Amazon S3 bucket as raw files. These files are then processed using AWS Glue ETL jobs to clean and transform the data into a structured format. An AWS Glue Crawler scans the data to infer schema and stores metadata in the AWS Glue Catalog. Once processed, the structured batch data is saved in a different S3 bucket. This data becomes queryable through Amazon Athena, which allows SQL-based analysis over S3. This layer enables retrospective analytics, such as trend analysis over several years of disease cases.

The Real-Time Layer, depicted at the bottom of the diagram, is responsible for processing live data streams such as real-time test results, hospital alert systems, or data from wearable health devices. Real-time data is ingested through Amazon Kinesis Data Streams. AWS Lambda functions are triggered upon new data arrival to transform and cleanse the stream before

sending it to Amazon Kinesis Data Firehose. Firehose stores the processed data in another S3 bucket and can optionally route data to analytics tools like Amazon Kinesis Data Analytics, which provides near real-time insights using streaming SQL queries. This layer enables detection of disease spikes and urgent events as they occur.

Between the batch and speed layers is the Merged Serving Layer. This layer merges historical batch data with real-time data to provide a comprehensive and unified dataset. Amazon EMR is used to run Spark or Hive jobs that integrate both types of data into a "merged view," which is stored in another S3 bucket or delivered to a data warehouse like Amazon Redshift or a relational database like RDS. Optionally, DynamoDB can be used to serve flexible, fast-access views of patient or disease-specific information. This unified layer allows downstream applications and dashboards to query data that reflects both long-term trends and recent developments, providing a full picture for decision-making.



The second diagram focuses on how the architecture is deployed in a secure, scalable, and fault-tolerant AWS infrastructure using best practices. The entire data processing framework is hosted within an Amazon Virtual Private Cloud (VPC), which provides network-level isolation and ensures that components are not directly exposed to the public internet.

To ensure high availability, the infrastructure is replicated across multiple Availability Zones (AZs) within a single AWS Region. These zones are independent data centers that operate in parallel. If one zone fails, traffic is automatically rerouted to the other zone without service disruption. This replication ensures both high availability and fault tolerance, which are critical for healthcare-related applications.

User and application traffic is managed through Amazon Route 53, which handles DNS routing, and an Elastic Load Balancer (ELB), which distributes traffic intelligently to the appropriate availability zone and resources for optimal performance. All components, including Lambda functions, EMR clusters, databases, and data streams, scale automatically depending on usage patterns through AWS Auto Scaling. For example, a sudden spike in real-time disease data due to an outbreak would automatically trigger more processing capacity without manual intervention.

Security is enforced through multiple layers. Identity and Access Management (IAM) ensures that only authorized users and services can access specific resources. AWS Key Management Service (KMS) provides encryption at rest, while all data in transit is secured using TLS/SSL encryption. Databases are deployed inside private subnets within the VPC, making them inaccessible from the public internet and reducing exposure to external threats.

Step 9: Snowflake Advantages Summary

Snowflake is a modern, fully managed cloud-based data warehouse designed for performance, scalability, and flexibility. It differs significantly from traditional RDBMS systems like PostgreSQL.

Benefits of Using Snowflake

1. Performance Without Tuning
    - Snowflake automatically handles indexing and partitioning
    - No need to manage VACUUM, ANALYZE, or tuning manually

2. Elastic Scalability
    - Can handle large volumes of test results, patient data, and historical records
    - Query performance remains consistent regardless of data size

3. Separation of Storage & Compute
    - You can pause compute when not querying — save cost
    - Useful for academic or proof-of-concept projects with burst usage

4. Concurrent Dashboards and Queries
    - Doctors, analysts, and researchers can run dashboards simultaneously without contention
    - BI tools like Tableau can connect directly to Snowflake via built-in connectors

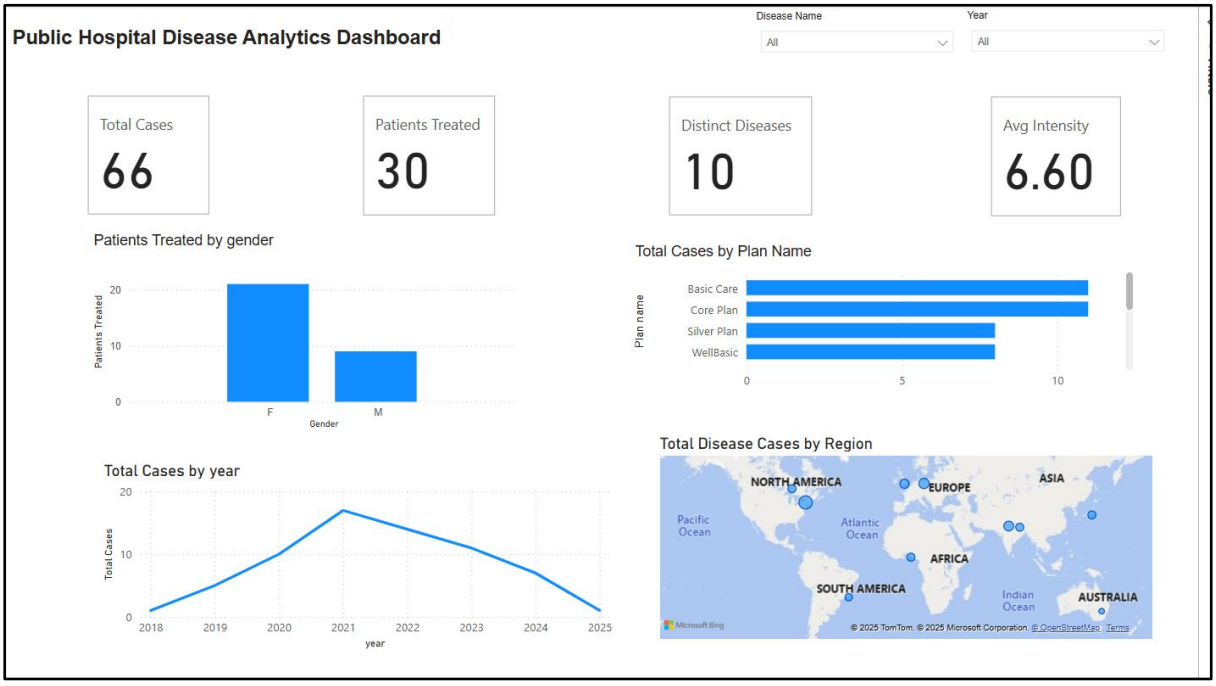
5. Data Sharing & Real-Time Integration
    - You could share disease trend data securely with government or hospitals without ETL
    - Snowpipe can load new test results in real-time

Summary of Part 9

While PostgreSQL serves well as a traditional relational database for OLTP and dimensional modeling, Snowflake offers significant advantages for large-scale analytical workloads. Its ability to scale compute and storage independently, handle concurrency, and simplify performance tuning makes it a highly attractive platform for healthcare analytics, especially when working with real-time or historical patient data at scale.

Dashboard:

File name: updatedimtables.sql (More data were inserted in between to make the dashboard clear and realistic.)



This dashboard, titled "Public Hospital Disease Analytics Dashboard", provides healthcare providers with a comprehensive overview of key metrics related to disease cases and treatment statistics. Below is a detailed breakdown of each section:

1. Key Metrics Overview (Top Row):

- Total Cases (66): The total number of disease cases recorded in the system.
- Patients Treated (30): The number of patients who have received treatment.
- Distinct Diseases (10): The number of unique diseases documented in the dataset.
- Average Intensity (6.60): The average severity level of the diseases, measured on a scale (likely from 1 to 10).

These metrics give healthcare providers a quick snapshot of the overall situation, including the volume of cases, treatment coverage, diversity of diseases, and average disease severity.

2. Patients Treated by Gender (Bar Chart):

- This bar chart shows the number of patients treated, categorized by gender (Female - "F" and Male - "M").
- Observation: A significantly higher number of female patients (around 20) compared to male patients (around 10), indicating a possible gender disparity in healthcare access or disease prevalence.

3. Total Cases by Plan Name (Horizontal Bar Chart):

- This chart displays the number of cases categorized by insurance plan or care plan name.
- The plans listed include:
    - Basic Care
    - Core Plan
    - Silver Plan
    - WellBasic
- Observation: The Basic Care plan has the highest number of cases, followed by other plans, indicating it may be the most commonly used or most accessible plan.

4. Total Cases by Year (Line Chart):

- This line graph displays the trend of total cases over time, from 2018 to 2025.
- Observation: A notable increase in cases from 2018, peaking around 2021, and then a gradual decline towards 2025. This trend may indicate the effectiveness of disease prevention measures or changes in healthcare practices.

5. Total Disease Cases by Region (Map):

- An interactive map visualizing the geographical distribution of disease cases.
- Observation: The map shows cases spread across major regions, including North America, Europe, Asia, Africa, South America, and Australia. This global spread suggests that the data might be aggregated from multiple healthcare facilities or global health monitoring systems.

Filters (Top Right):

- Disease Name and Year: Allows healthcare providers to filter the data based on a specific disease or year, enabling more targeted analysis.

Insights for Healthcare Providers:

1. Patient Demographics: Identifying gender disparities can help allocate resources better or initiate gender-specific healthcare programs.
2. Plan Utilization: Understanding which care plans are most used can guide policy adjustments or funding allocations.

3. Yearly Trends: Monitoring changes in total cases over the years can assist in evaluating the impact of public health interventions.
4. Geographical Insights: Knowing where diseases are most prevalent supports targeted healthcare initiatives in affected regions.

CONCLUSION

The Disease Data Management and Analytics System is designed to enhance public health outcomes by leveraging a robust relational database model coupled with a star schema to facilitate analytical insights. This system incorporates Slowly Changing Dimension Type 2 (SCD2) to ensure the accuracy of historical data, allowing for comprehensive tracking of changes over time. To support scalable and real-time data processing, the system utilizes AWS Lambda architecture, which enables the efficient handling of dynamic healthcare data. Additionally, it integrates Snowflake to optimize the performance and management of large-scale healthcare datasets, providing both speed and flexibility. By comparing relational databases with NoSQL alternatives, the system achieves a balanced approach, managing structured data while supporting flexible, real-time analytics. This comprehensive data management and analytics approach empowers public health agencies to make informed decisions, ultimately improving health outcomes through data-driven insights.